



SpringMVC 第一天

框架课程



1. 课程计划

第一天

- 1、SpringMVC 介绍
- 2、入门程序
- 3、SpringMVC 架构讲解
 - a) 框架结构
 - b) 组件说明
- 4、SpringMVC 整合 MyBatis
- 5、参数绑定
 - a) SpringMVC 默认支持的类型
 - b) 简单数据类型
 - c) Pojo 类型
 - d) Pojo 包装类型
 - e) 自定义参数绑定
- 6、SpringMVC 和 Struts2 的区别

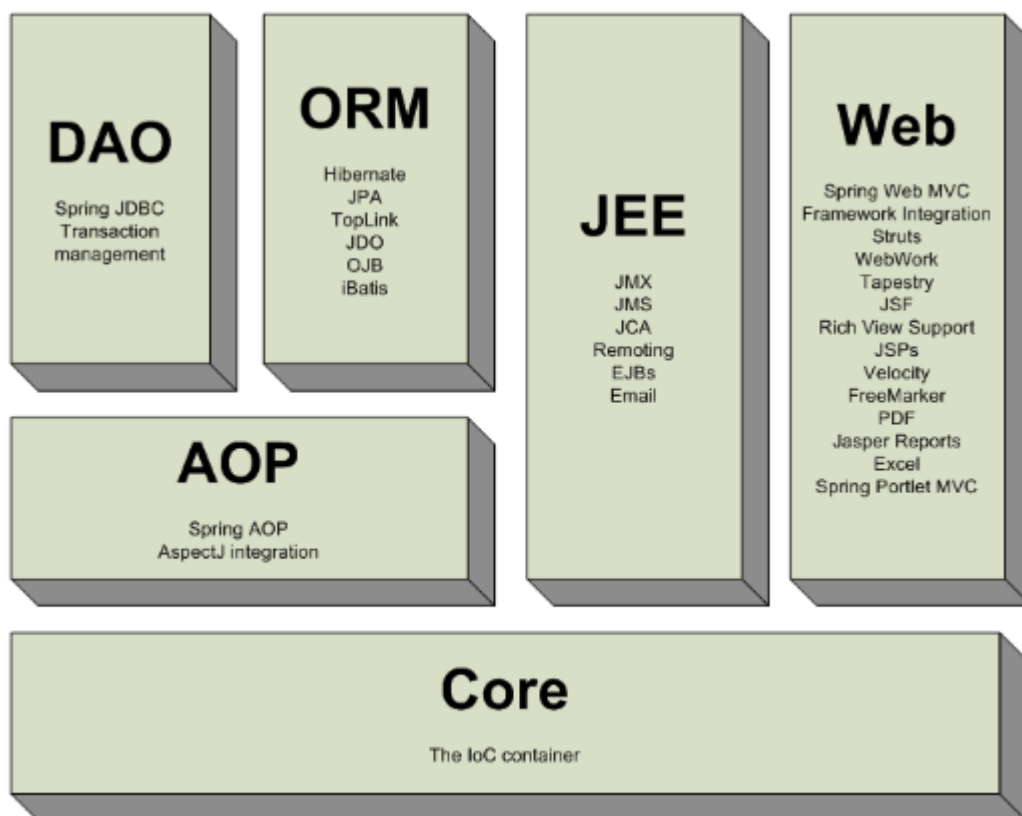
第二天

- 1、高级参数绑定
 - a) 数组类型的参数绑定
 - b) List 类型的绑定
- 2、@RequestMapping 注解的使用
- 3、Controller 方法返回值
- 4、SpringMVC 中异常处理
- 5、图片上传处理
- 6、Json 数据交互
- 7、SpringMVC 实现 RESTful
- 8、拦截器

2. Spring 入门

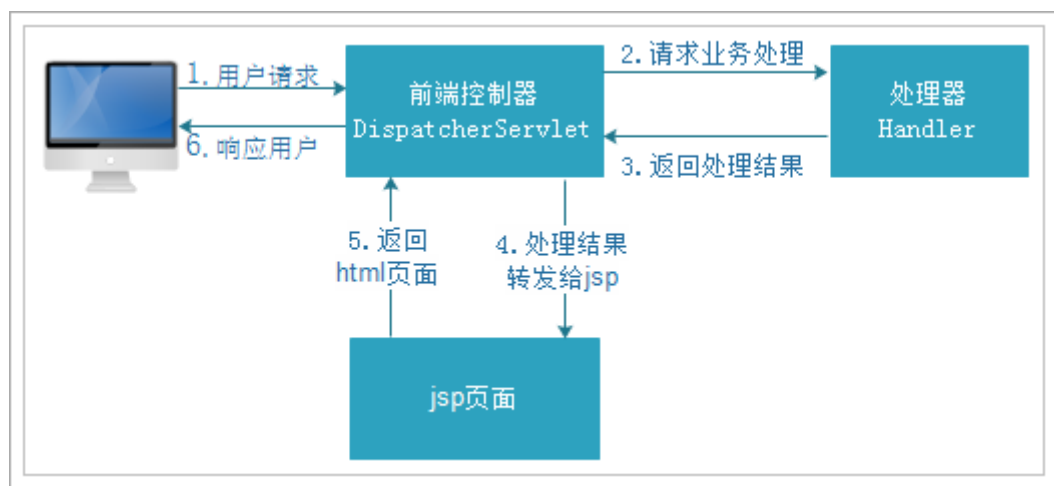
2.1. Springmvc 是什么

Spring web mvc 和 Struts2 都属于表现层的框架,它是 Spring 框架的一部分,我们可以从 Spring 的整体结构中看得出来,如下图:



2.2. Springmvc 处理流程

如下图所示:



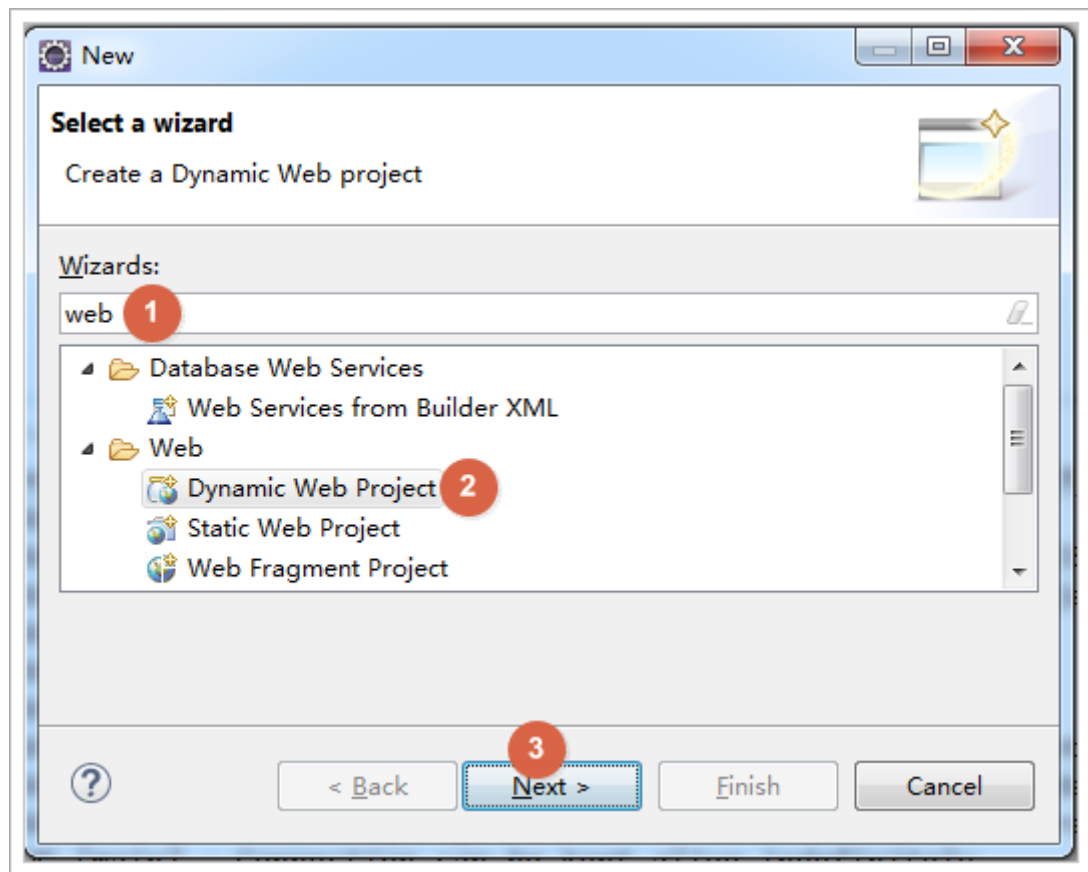
2.3. 入门程序

需求：使用浏览器显示商品列表

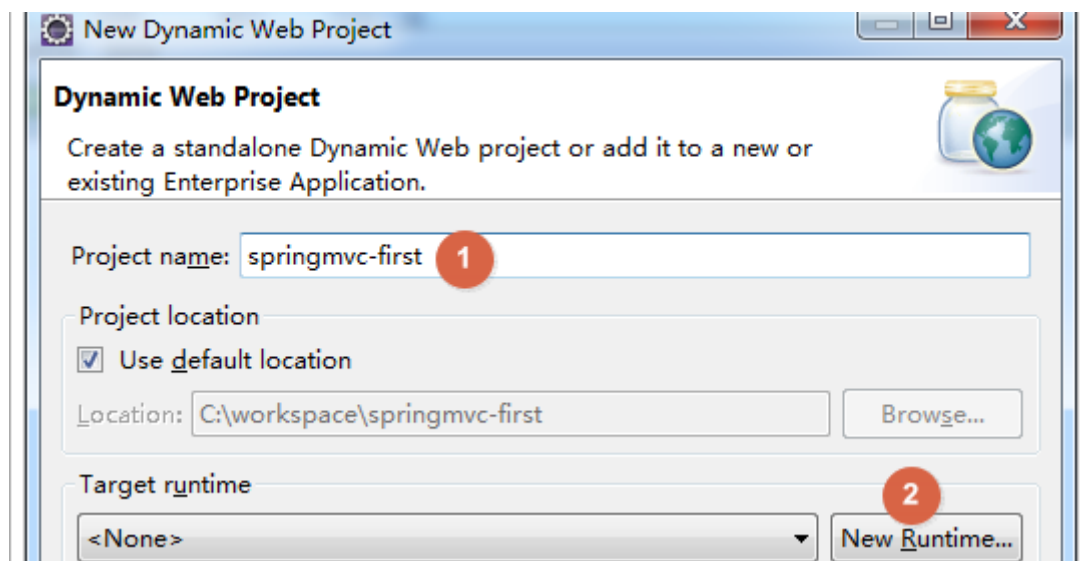
2.3.1. 创建 web 工程

springMVC 是表现层框架，需要搭建 web 工程开发。

如下图创建动态 web 工程：

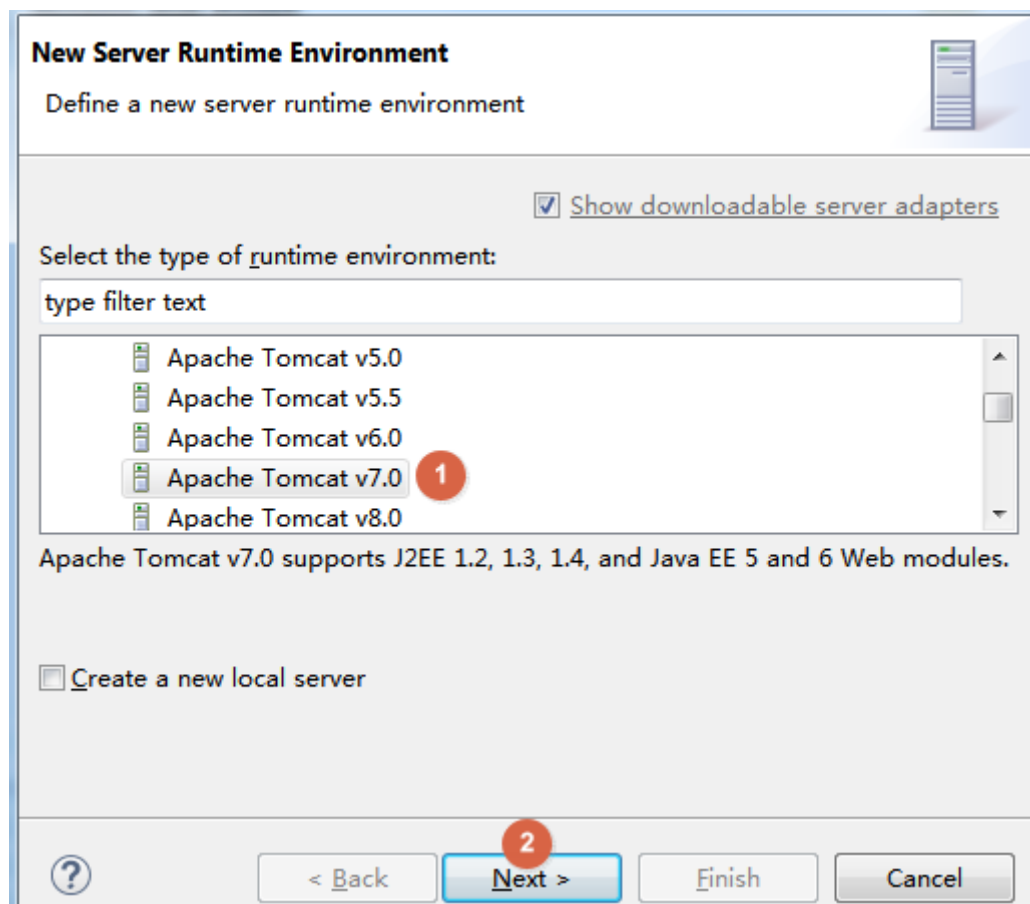


输入工程名，选择配置 Tomcat（如果已有，则直接使用），如下图：

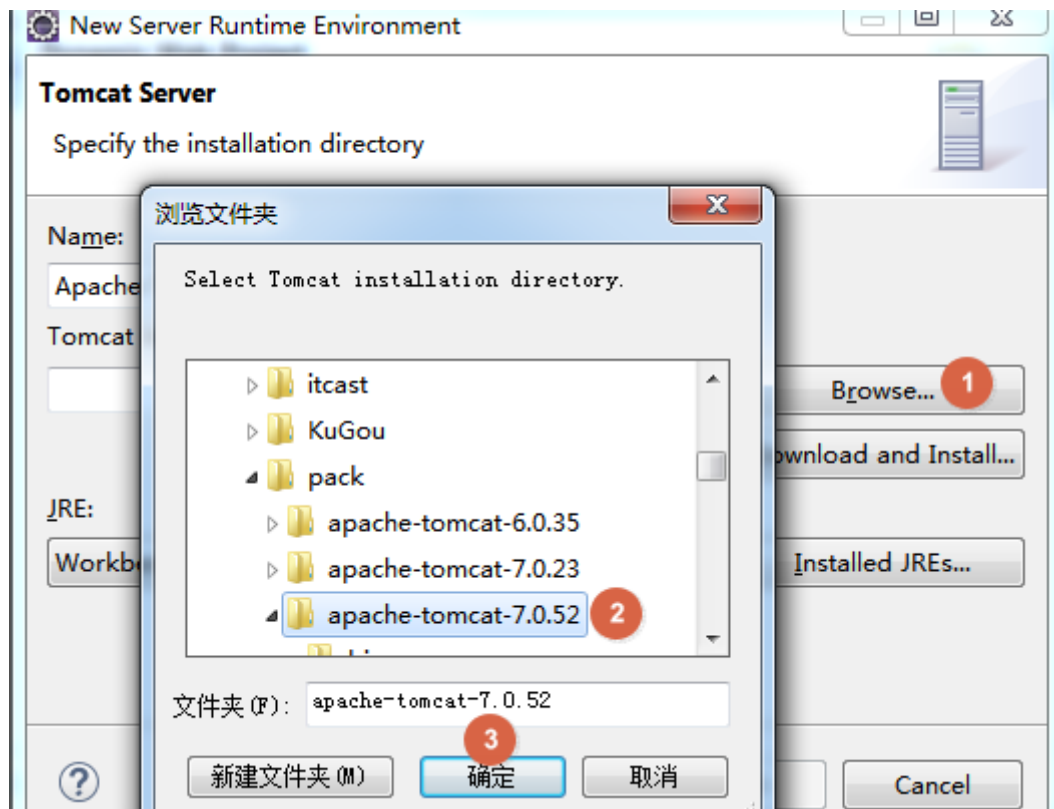




配置 Tomcat，如下图：

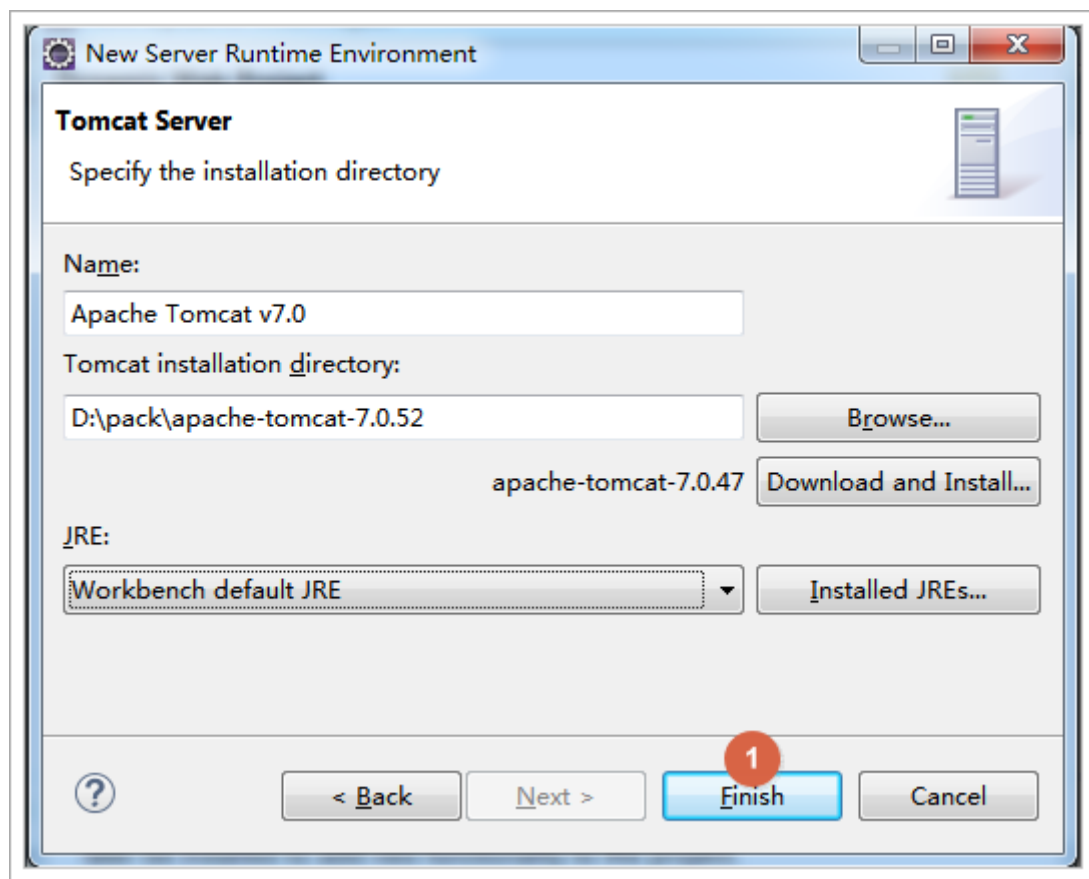


选择准备好的 Tomcat，这里用的是 Tomcat7，如下图：

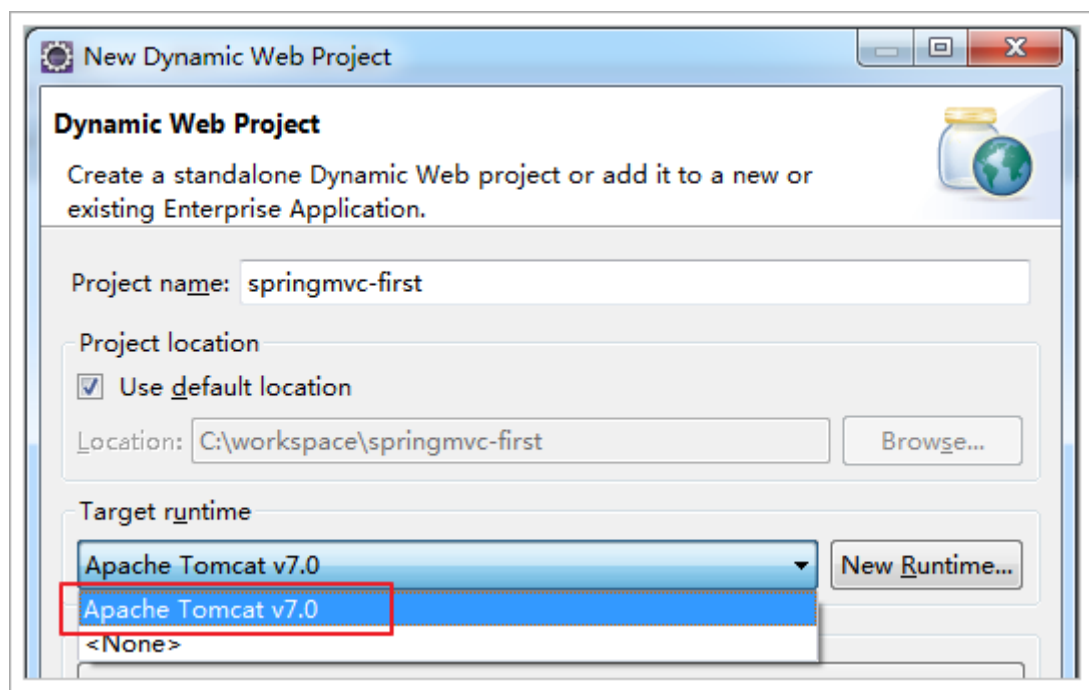




选择成功，点击 Finish，如下图：

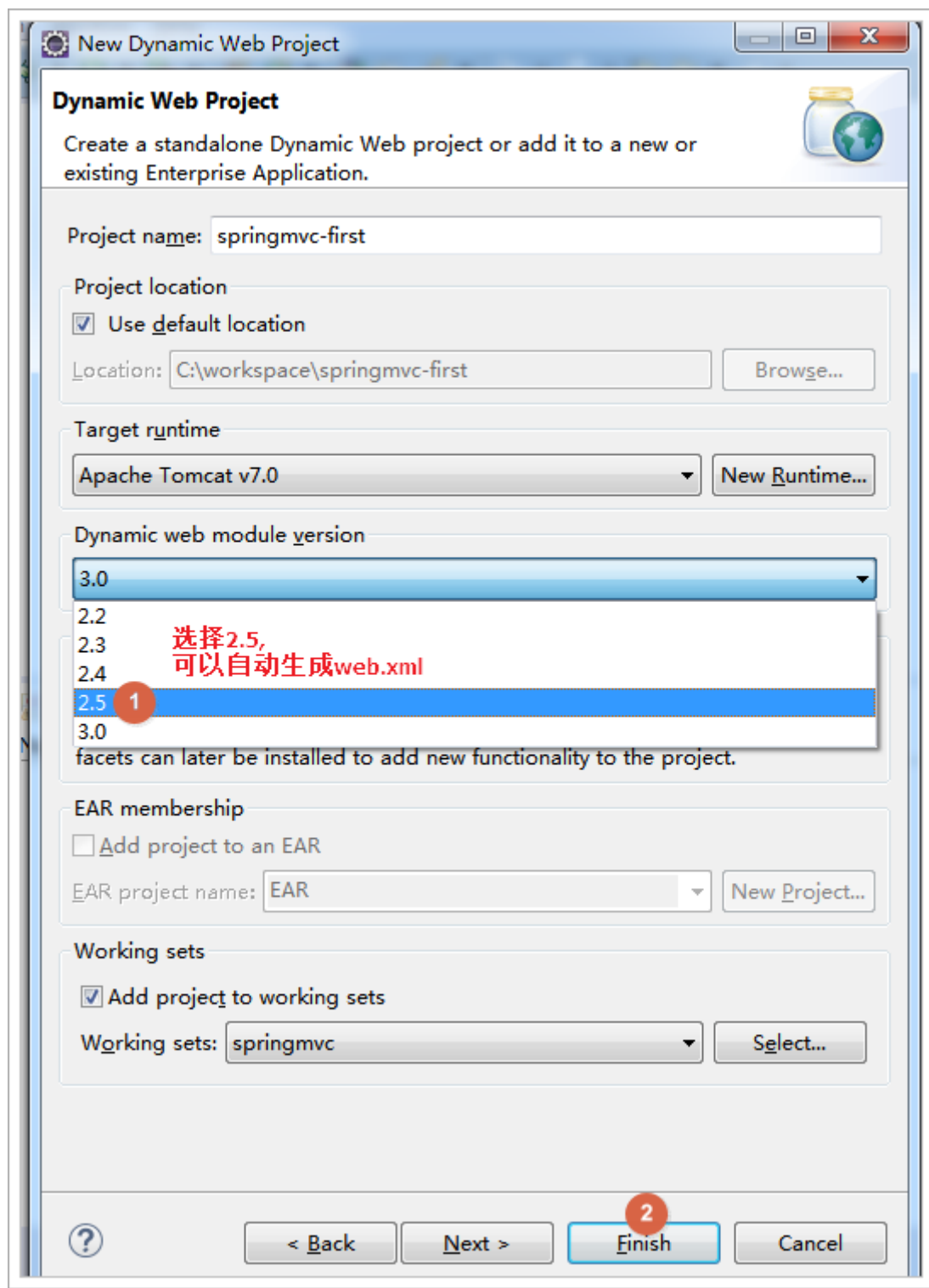


选择刚刚设置成功的 Tomcat，如下图：

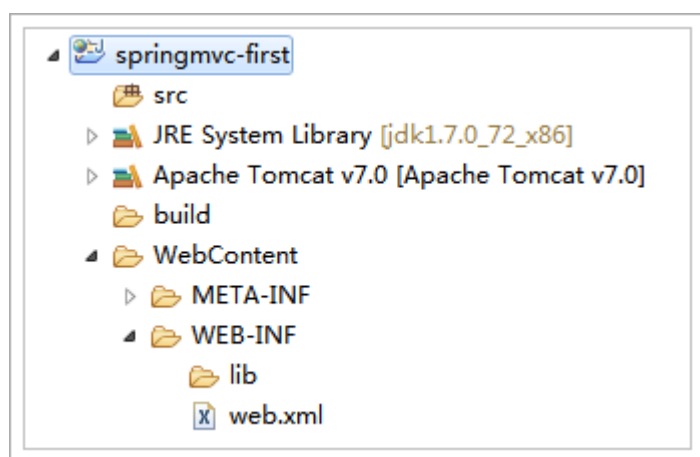




如下图选择 web 的版本是 2.5，可以自动生成 web.xml 配置文件，

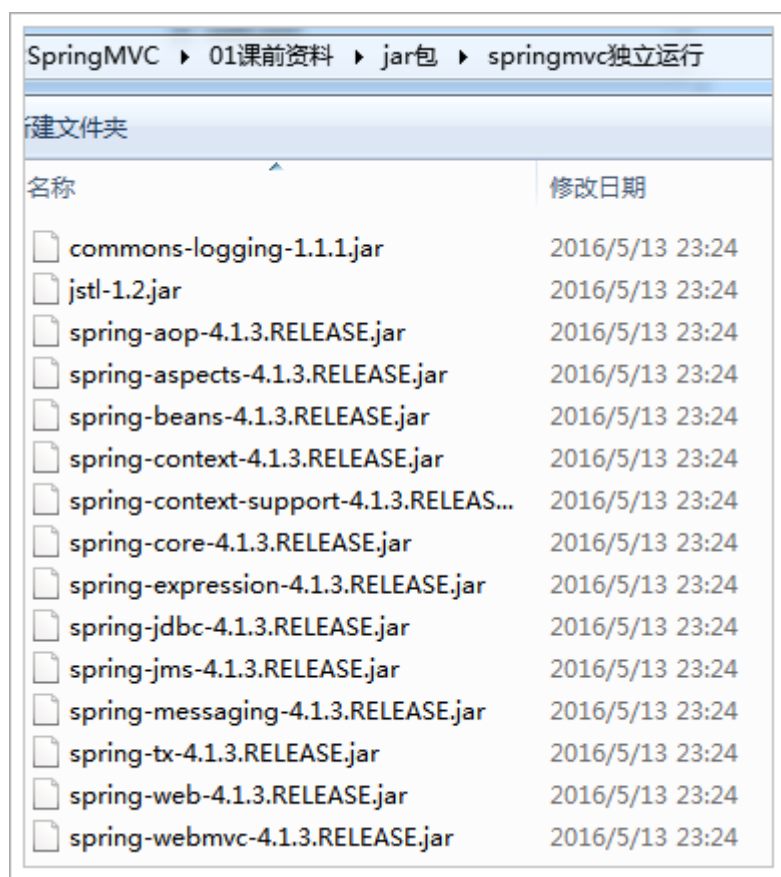


创建效果如下图：

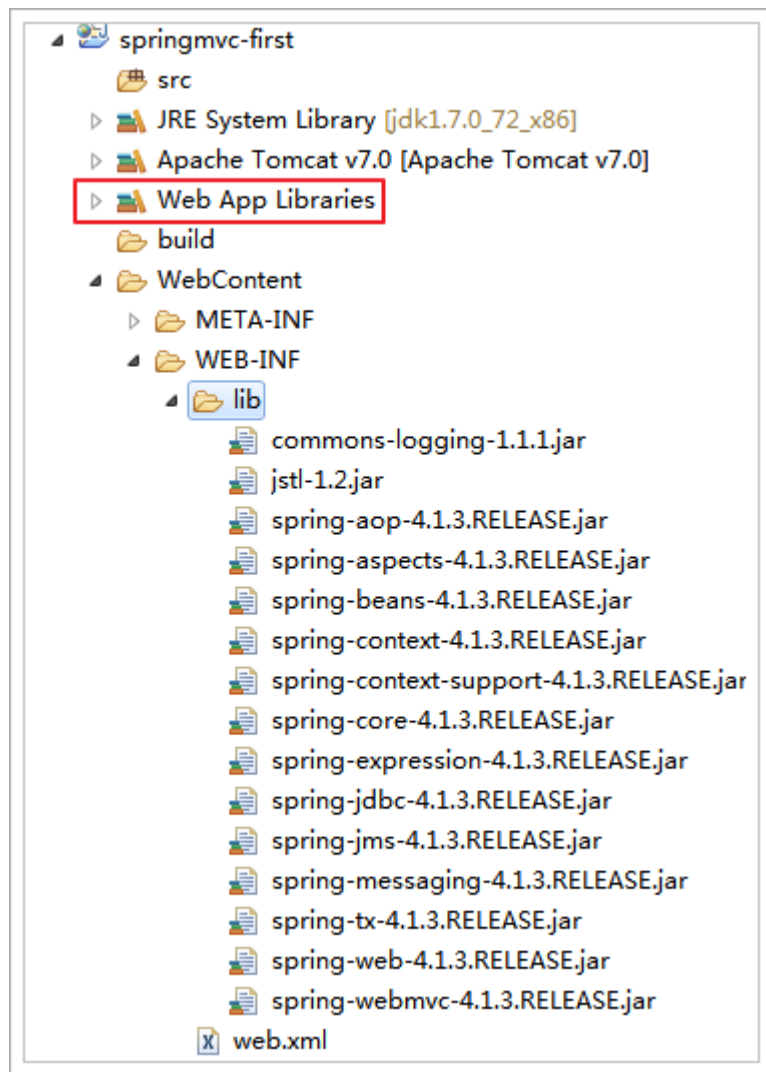


2.3.2. 导入 jar 包

从课前资料中导入 springMVC 的 jar 包，位置如下图：

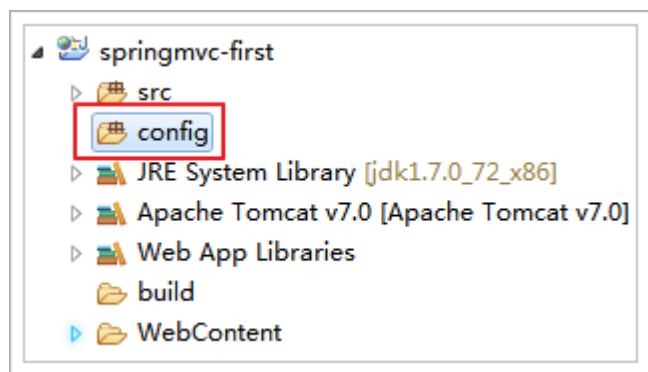


复制 jar 到 lib 目录，工程直接加载 jar 包，如下图：



2.3.3. 加入配置文件

创建 config 资源文件夹，存放配置文件，如下图：





2.3.3.1. 创建 springmvc.xml

创建 SpringMVC 的核心配置文件

SpringMVC 本身就是 Spring 的子项目,对 Spring 兼容性很好,不需要做很多配置。这里只配置一个 Controller 扫描就可以了,让 Spring 对页面控制层 Controller 进行管理。

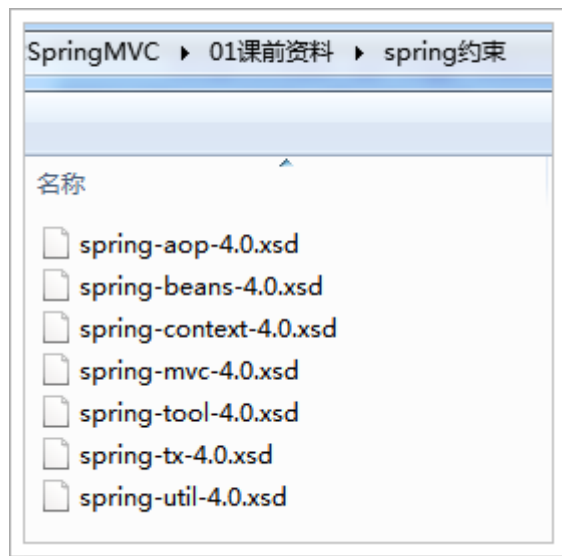
创建 springmvc.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:p="http://www.springframework.org/schema/p"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:mvc="http://www.springframework.org/schema/mvc"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
http://www.springframework.org/schema/mvc
http://www.springframework.org/schema/mvc/spring-mvc-4.0.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context-4.0.xsd"
>

    <!-- 配置 controller 扫描包 -->
    <context:component-scan
base-package="cn.itcast.springmvc.controller" />

</beans>
```

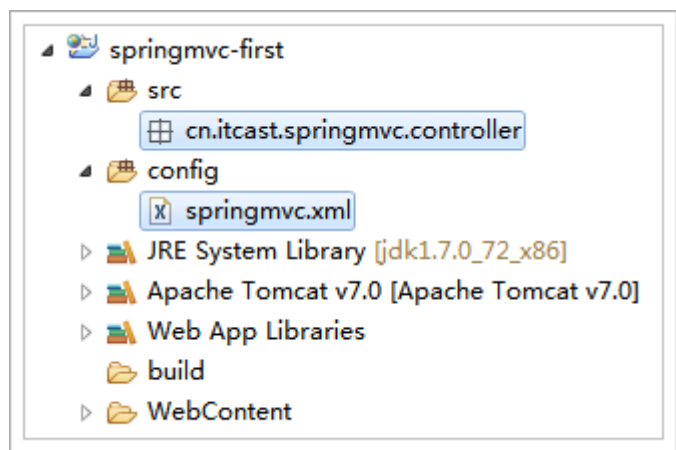
配置文件需要的约束文件,位置如下图:





创建包 cn.itcast.springmvc.controller

效果如下图：



2.3.3.2. 配置前端控制器

配置 SpringMVC 的前端控制器 DispatcherServlet

在 web.xml 中

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="http://java.sun.com/xml/ns/javaee"
  xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
  id="WebApp_ID" version="2.5">
  <display-name>springmvc-first</display-name>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
    <welcome-file>index.htm</welcome-file>
    <welcome-file>index.jsp</welcome-file>
    <welcome-file>default.html</welcome-file>
    <welcome-file>default.htm</welcome-file>
    <welcome-file>default.jsp</welcome-file>
  </welcome-file-list>

  <!-- 配置 SpringMVC 前端控制器 -->
  <servlet>
    <servlet-name>springmvc-first</servlet-name>

    <servlet-class>org.springframework.web.servlet.DispatcherServlet<
/servlet-class>
    <!-- 指定 SpringMVC 配置文件 -->
    <!-- SpringMVC 的配置文件的默认路径是
/WEB-INF/${servlet-name}-servlet.xml -->
```

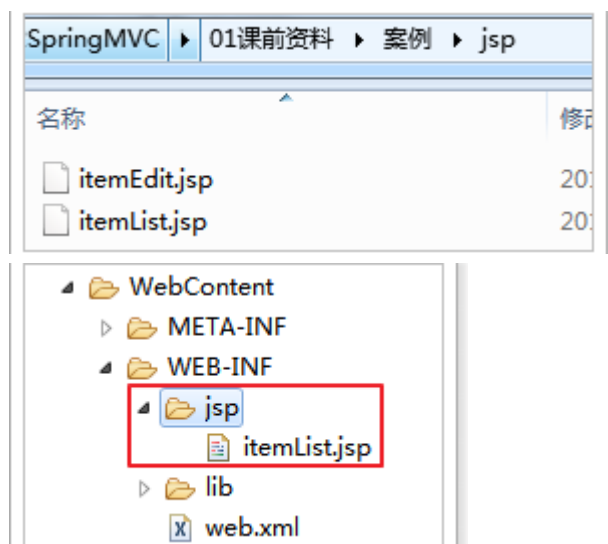


```
<init-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>classpath:springmvc.xml</param-value>
</init-param>
</servlet>

<servlet-mapping>
    <servlet-name>springmvc-first</servlet-name>
    <!-- 设置所有以 action 结尾的请求进入 SpringMVC -->
    <url-pattern>*.action</url-pattern>
</servlet-mapping>
</web-app>
```

2.3.4. 加入 jsp 页面

把参考资料中的 itemList.jsp 复制到工程的/WEB-INF/jsp 目录下，如下图：



2.3.5. 实现显示商品列表页

2.3.5.1. 创建 pojo

分析页面，查看页面需要的数据，如下图：





创建商品 pojo

```
public class Item {  
    // 商品 id  
    private int id;  
    // 商品名称  
    private String name;  
    // 商品价格  
    private double price;  
    // 商品创建时间  
    private Date createtime;  
    // 商品描述  
    private String detail;  
}
```

创建带参数的构造器

```
set/get...  
}
```

2.3.5.2. 创建 ItemController

ItemController 是一个普通的 java 类，不需要实现任何接口。

需要在类上添加@Controller 注解，把 Controller 交由 Spring 管理

在方法上面添加@RequestMapping 注解，里面指定请求的 url。其中“.action”可以加也可以不加。

```
@Controller  
public class ItemController {  
  
    // @RequestMapping: 里面放的是请求的 url，和用户请求的 url 进行匹配  
    // action 可以写也可以不写  
    @RequestMapping("/itemList.action")  
    public ModelAndView queryItemList() {  
        // 创建页面需要显示的商品数据  
        List<Item> list = new ArrayList<>();  
        list.add(new Item(1, "1 华为 荣耀 8", 2399, new Date(), "质量好! 1"));  
        list.add(new Item(2, "2 华为 荣耀 8", 2399, new Date(), "质量好! 2"));  
        list.add(new Item(3, "3 华为 荣耀 8", 2399, new Date(), "质量好! 3"));  
        list.add(new Item(4, "4 华为 荣耀 8", 2399, new Date(), "质量好! 4"));  
        list.add(new Item(5, "5 华为 荣耀 8", 2399, new Date(), "质量好! 5"));  
        list.add(new Item(6, "6 华为 荣耀 8", 2399, new Date(), "质量好! 6"));  
  
        // 创建 ModelAndView，用来存放数据和视图  
        ModelAndView modelAndView = new ModelAndView();  
        // 设置数据到模型中
```



```
modelAndView.addObject("list", list);  
// 设置视图 jsp，需要设置视图的物理地址  
modelAndView.setViewName("/WEB-INF/jsp/itemList.jsp");  
  
return modelAndView;  
}  
}
```

2.3.6. 启动项目测试

启动项目，浏览器访问地址

http://127.0.0.1:8080/springmvc-first/itemList.action

效果如下图：

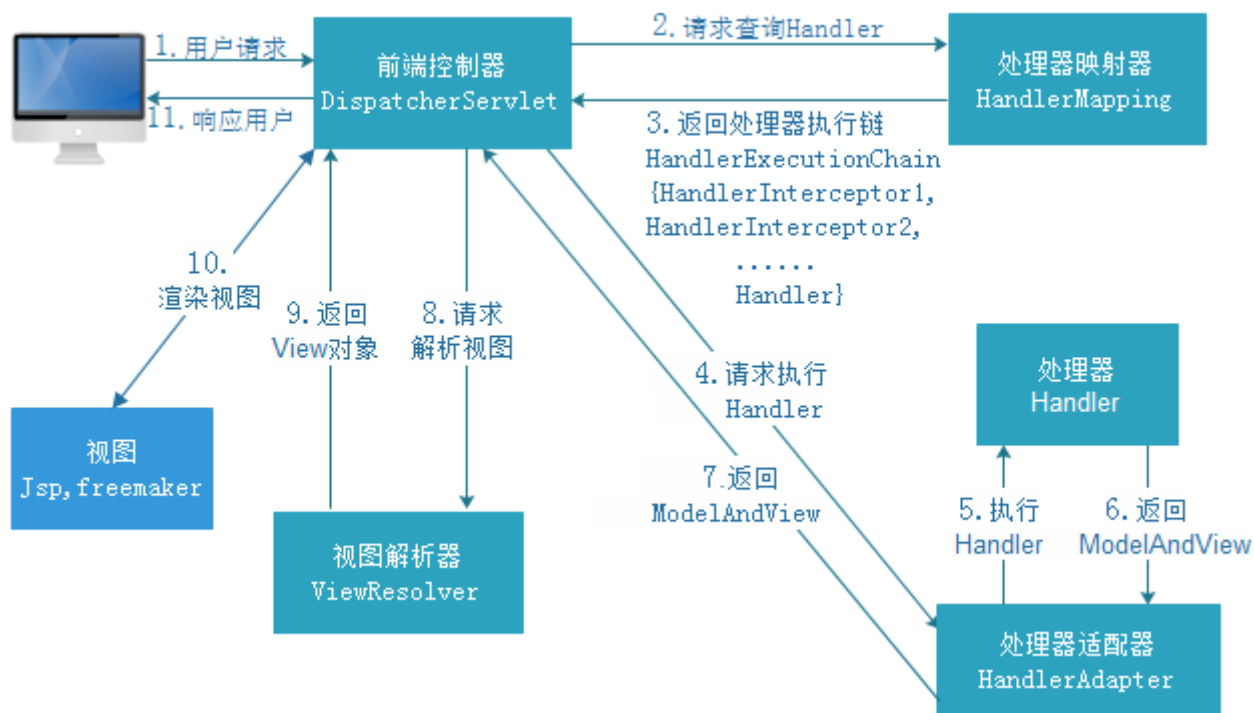
商品名称	商品价格	生产日期	商品描述	操作
1华为 荣耀8	2399.0	2016-07-29 21:04:43	质量好! 1	修改
2华为 荣耀8	2399.0	2016-07-29 21:04:43	质量好! 2	修改
3华为 荣耀8	2399.0	2016-07-29 21:04:43	质量好! 3	修改
4华为 荣耀8	2399.0	2016-07-29 21:04:43	质量好! 4	修改
5华为 荣耀8	2399.0	2016-07-29 21:04:43	质量好! 5	修改
6华为 荣耀8	2399.0	2016-07-29 21:04:43	质量好! 6	修改

为什么可以用呢？我们需要分析一下 springMVC 的架构图。

3. Springmvc 架构

3.1. 框架结构

框架结构如下图：



3.2. 架构流程

- 1、用户发送请求至前端控制器DispatcherServlet
- 2、DispatcherServlet收到请求调用HandlerMapping处理器映射器。
- 3、处理器映射器根据请求url找到具体的处理器，生成处理器对象及处理器拦截器(如果有则生成)一并返回给DispatcherServlet。
- 4、DispatcherServlet通过HandlerAdapter处理器适配器调用处理器
- 5、执行处理器(Controller，也叫后端控制器)。
- 6、Controller执行完成返回ModelAndView
- 7、HandlerAdapter将controller执行结果ModelAndView返回给DispatcherServlet
- 8、DispatcherServlet将ModelAndView传给ViewReslover视图解析器
- 9、ViewReslover解析后返回具体View
- 10、DispatcherServlet对View进行渲染视图(即将模型数据填充至视图中)。
- 11、DispatcherServlet响应用户



3.3. 组件说明

以下组件通常使用框架提供实现：

◆ DispatcherServlet：前端控制器

用户请求到达前端控制器，它就相当于mvc模式中的c，dispatcherServlet是整个流程控制的中心，由它调用其它组件处理用户的请求，dispatcherServlet的存在降低了组件之间的耦合性。

◆ HandlerMapping：处理器映射器

HandlerMapping负责根据用户请求url找到Handler即处理器，springmvc提供了不同的映射器实现不同的映射方式，例如：配置文件方式，实现接口方式，注解方式等。

◆ Handler：处理器

Handler 是继DispatcherServlet前端控制器的后端控制器，在DispatcherServlet的控制下Handler对具体的用户请求进行处理。

由于Handler涉及到具体的用户业务请求，所以一般情况需要程序员根据业务需求开发Handler。

◆ HandlerAdapter：处理器适配器

通过HandlerAdapter对处理器进行执行，这是适配器模式的应用，通过扩展适配器可以对更多类型的处理器进行执行。

下图是许多不同的适配器，最终都可以使用usb接口连接



◆ ViewResolver：视图解析器

View Resolver负责将处理结果生成View视图，View Resolver首先根据逻辑视图名解析成物理视图名即具体的页面地址，再生成View视图对象，最后对View进行渲染将处理结果通过页面展示给用户。

◆ View：视图

springmvc框架提供了很多的View视图类型的支持，包括：jstlView、freemarkerView、pdfView等。我们最常用的视图就是jsp。

一般情况下需要通过页面标签或页面模版技术将模型数据通过页面展示给用户，需要由程序员根据业务需求开发具体的页面。



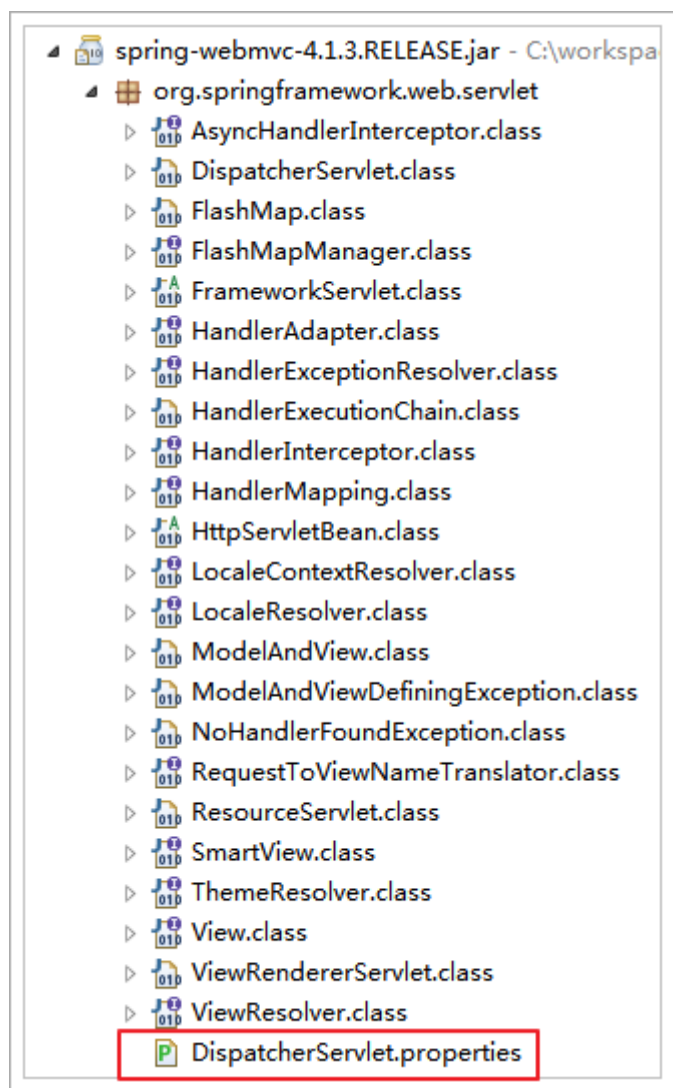
说明：在 `springmvc` 的各个组件中，处理器映射器、处理器适配器、视图解析器称为 `springmvc` 的三大组件。

需要用户开发的组件有 `handler`、`view`

3.4. 默认加载的组件

我们没有做任何配置，就可以使用这些组件

因为框架已经默认加载这些组件了，配置文件位置如下图：



```
# Default implementation classes for DispatcherServlet's strategy  
interfaces.  
# Used as fallback when no matching beans are found in the  
DispatcherServlet context.  
# Not meant to be customized by application developers.
```



```
org.springframework.web.servlet.LocaleResolver=org.springframework.w
eb.servlet.i18n.AcceptHeaderLocaleResolver

org.springframework.web.servlet.ThemeResolver=org.springframework.we
b.servlet.theme.FixedThemeResolver

org.springframework.web.servlet.HandlerMapping=org.springframework.w
eb.servlet.handler.BeanNameUrlHandlerMapping,\
    org.springframework.web.servlet.mvc.annotation.DefaultAnnotationH
andlerMapping

org.springframework.web.servlet.HandlerAdapter=org.springframework.w
eb.servlet.mvc.HttpRequestHandlerAdapter,\
    org.springframework.web.servlet.mvc.SimpleControllerHandlerAdapte
r,\
    org.springframework.web.servlet.mvc.annotation.AnnotationMethodHa
ndlerAdapter

org.springframework.web.servlet.HandlerExceptionResolver=org.springf
ramework.web.servlet.mvc.annotation.AnnotationMethodHandlerException
Resolver,\
    org.springframework.web.servlet.mvc.annotation.ResponseStatusExce
ptionResolver,\
    org.springframework.web.servlet.mvc.support.DefaultHandlerExcepti
onResolver

org.springframework.web.servlet.RequestToViewNameTranslator=org.spr
ingframework.web.servlet.view.DefaultRequestToViewNameTranslator

org.springframework.web.servlet.ViewResolver=org.springframework.web
.servlet.view.InternalResourceViewResolver

org.springframework.web.servlet.FlashMapManager=org.springframework.
web.servlet.support.SessionFlashMapManager
```

3.5. 组件扫描器

使用组件扫描器省去在 spring 容器配置每个 Controller 类的繁琐。
使用<context:component-scan>自动扫描标记@Controller 的控制器类，
在 springmvc.xml 配置文件中配置如下：

```
<!-- 配置 controller 扫描包，多个包之间用,分隔 -->
<context:component-scan base-package="cn.itcast.springmvc.controller"
/>
```



3.6. 注解映射器和适配器

3.6.1. 配置处理器映射器

注解式处理器映射器，对类中标记了 `@RequestMapping` 的方法进行映射。根据 `@RequestMapping` 定义的 url 匹配 `@RequestMapping` 标记的方法，匹配成功返回 `HandlerMethod` 对象给前端控制器。

`HandlerMethod` 对象中封装 url 对应的方法 `Method`。

从 spring3.1 版本开始，废除了 `DefaultAnnotationHandlerMapping` 的使用，推荐使用 `RequestMappingHandlerMapping` 完成注解式处理器映射。

在 `springmvc.xml` 配置文件中配置如下：

```
<!-- 配置处理器映射器 -->
<bean
    class="org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerMapping" />
```

注解描述：

@RequestMapping: 定义请求 url 到处理器功能方法的映射

3.6.2. 配置处理器适配器

注解式处理器适配器，对标记 `@RequestMapping` 的方法进行适配。

从 spring3.1 版本开始，废除了 `AnnotationMethodHandlerAdapter` 的使用，推荐使用 `RequestMappingHandlerAdapter` 完成注解式处理器适配。

在 `springmvc.xml` 配置文件中配置如下：

```
<!-- 配置处理器适配器 -->
<bean
    class="org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdapter" />
```



3.6.3. 注解驱动

直接配置处理器映射器和处理器适配器比较麻烦，可以使用注解驱动来加载。

SpringMVC 使用<mvc:annotation-driven>自动加载 RequestMappingHandlerMapping 和 RequestMappingHandlerAdapter

可以在 springmvc.xml 配置文件中使用<mvc:annotation-driven>替代注解处理器和适配器的配置。

```
<!-- 注解驱动 -->
<mvc:annotation-driven />
```

3.7. 视图解析器

视图解析器使用 SpringMVC 框架默认的 InternalResourceViewResolver，这个视图解析器支持 JSP 视图解析

在 springmvc.xml 配置文件中配置如下：

```
<!-- Example: prefix="/WEB-INF/jsp/", suffix=".jsp",
viewname="test" ->
"/WEB-INF/jsp/test.jsp" -->
<!-- 配置视图解析器 -->
<bean

    class="org.springframework.web.servlet.view.InternalResourceViewR
esolver">
    <!-- 配置逻辑视图的前缀 -->
    <property name="prefix" value="/WEB-INF/jsp/" />
    <!-- 配置逻辑视图的后缀 -->
    <property name="suffix" value=".jsp" />
</bean>
```

逻辑视图名需要在controller中返回 ModelAndView 指定，比如逻辑视图名为 ItemList，则最终返回的jsp视图地址：

“WEB-INF/jsp/itemList.jsp”

最终 jsp 物理地址：前缀+逻辑视图名+后缀

3.7.1. 修改 ItemController

修改 ItemController 中设置视图的代码

```
// @RequestMapping: 里面放的是请求的 url，和用户请求的 url 进行匹配
// action 可以写也可以不写
@RequestMapping("/itemList.action")
```



```
public ModelAndView queryItemList() {  
    // 创建页面需要显示的商品数据  
    List<Item> list = new ArrayList<>();  
    list.add(new Item(1, "1 华为 荣耀 8", 2399, new Date(), "质量好! 1"));  
    list.add(new Item(2, "2 华为 荣耀 8", 2399, new Date(), "质量好! 2"));  
    list.add(new Item(3, "3 华为 荣耀 8", 2399, new Date(), "质量好! 3"));  
    list.add(new Item(4, "4 华为 荣耀 8", 2399, new Date(), "质量好! 4"));  
    list.add(new Item(5, "5 华为 荣耀 8", 2399, new Date(), "质量好! 5"));  
    list.add(new Item(6, "6 华为 荣耀 8", 2399, new Date(), "质量好! 6"));  
  
    // 创建 ModelAndView，用来存放数据和视图  
    ModelAndView modelAndView = new ModelAndView();  
    // 设置数据到模型中  
    modelAndView.addObject("itemList", list);  
    // 设置视图 jsp，需要设置视图的物理地址  
    // modelAndView.setViewName("/WEB-INF/jsp/itemList.jsp");  
    // 配置好视图解析器前缀和后缀，这里只需要设置逻辑视图就可以了。  
    // 视图解析器根据前缀+逻辑视图名+后缀拼接出来物理路径  
    modelAndView.setViewName("itemList");  
  
    return modelAndView;  
}
```

3.7.2. 效果

效果和之前一样，如下图：

查询条件:

商品列表:

商品名称	商品价格	生产日期	商品描述	操作
1华为 荣耀8	2399.0	2016-07-29 21:04:43	质量好! 1	修改
2华为 荣耀8	2399.0	2016-07-29 21:04:43	质量好! 2	修改
3华为 荣耀8	2399.0	2016-07-29 21:04:43	质量好! 3	修改
4华为 荣耀8	2399.0	2016-07-29 21:04:43	质量好! 4	修改
5华为 荣耀8	2399.0	2016-07-29 21:04:43	质量好! 5	修改
6华为 荣耀8	2399.0	2016-07-29 21:04:43	质量好! 6	修改

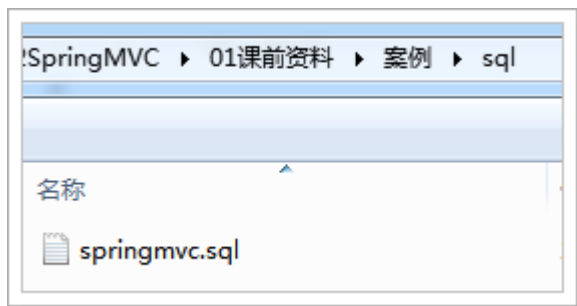
4. 整合 mybatis

为了更好的学习 springmvc 和 mybatis 整合开发的方法，需要将 springmvc 和 mybatis 进行整合。

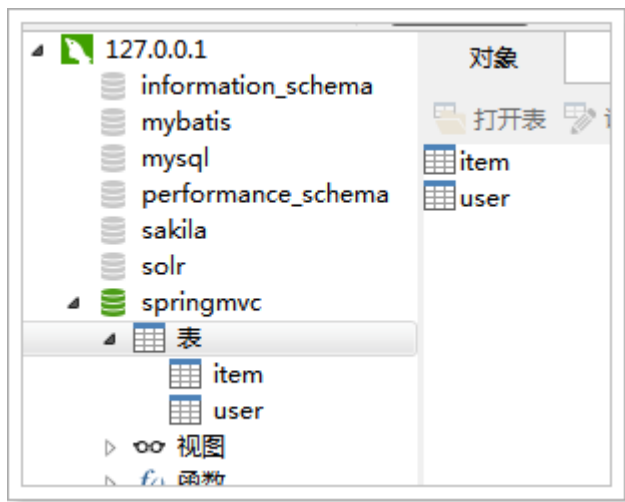
整合目标：控制层采用 springmvc、持久层使用 mybatis 实现。

4.1. 创建数据库表

sql 脚本，位置如下图：



创建数据库表 springmvc，导入到数据库中，如下图：



4.2. 需要的 jar 包

1. spring（包括 springmvc）
2. mybatis
3. mybatis-spring 整合包
4. 数据库驱动
5. 第三方连接池。



jar 包位置如下图：



4.3. 整合思路

Dao 层：

- 1、SqlMapConfig.xml，空文件即可，但是需要文件头。
- 2、applicationContext-dao.xml
 - a) 数据库连接池
 - b) SqlSessionFactory 对象，需要 spring 和 mybatis 整合包下的。
 - c) 配置 mapper 文件扫描器。

Service 层：

- 1、applicationContext-service.xml 包扫描器，扫描@Service 注解的类。
- 2、applicationContext-trans.xml 配置事务。

Controller 层：

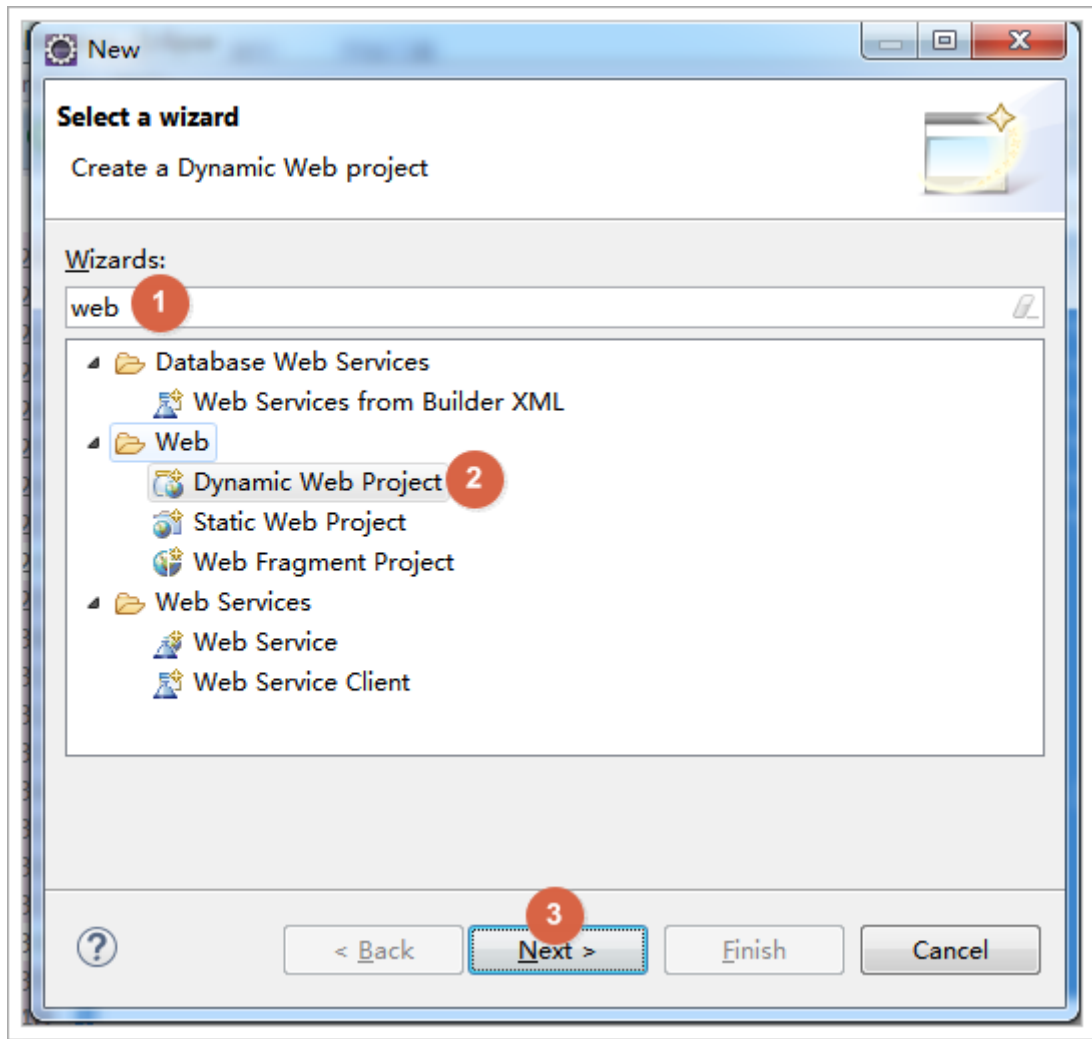
- 1、Springmvc.xml
 - a) 包扫描器，扫描@Controller 注解的类。
 - b) 配置注解驱动
 - c) 配置视图解析器

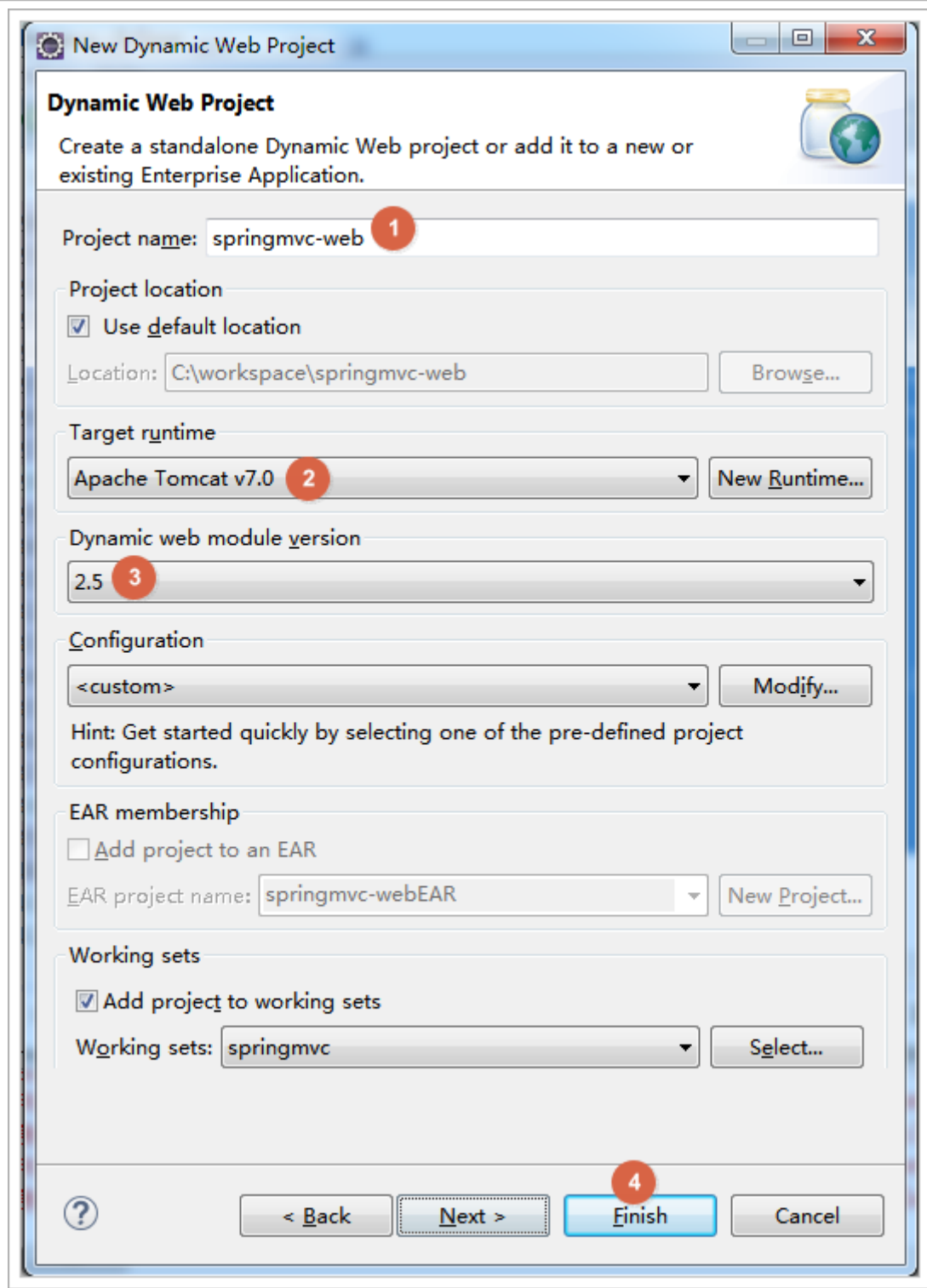
Web.xml 文件：

- 1、配置 spring
- 2、配置前端控制器。

4.4. 创建工程

创建动态 web 工程 springmvc-web，如下图：





4.5. 加入 jar 包

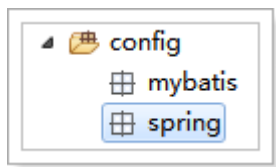
复制 jar 包到/WEB-INF/lib 中
工程自动加载 jar 包



4.6. 加入配置文件

创建资源文件夹 config

在其下创建 mybatis 和 spring 文件夹，用来存放配置文件，如下图：



4.6.1. sqlMapConfig.xml

使用逆向工程来生成 Mapper 相关代码，不需要配置别名。

在 config/mybatis 下创建 SqlMapConfig.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE configuration
PUBLIC "-//mybatis.org//DTD Config 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-config.dtd">
<configuration>

</configuration>
```

4.6.2. applicationContext-dao.xml

配置数据源、配置 SqlSessionFactory、mapper 扫描器。

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:p="http://www.springframework.org/schema/p"
    xmlns:aop="http://www.springframework.org/schema/aop"
    xmlns:tx="http://www.springframework.org/schema/tx"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context-4.0.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop-4.0.xsd
http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx-4.0.xsd
http://www.springframework.org/schema/util
http://www.springframework.org/schema/util/spring-util-4.0.xsd">

<!-- 加载配置文件 -->
```



```
<context:property-placeholder location="classpath:db.properties"
/>

<!-- 数据库连接池 -->
<bean id="dataSource"
class="org.apache.commons.dbcp.BasicDataSource"
destroy-method="close">
    <property name="driverClassName" value="${jdbc.driver}" />
    <property name="url" value="${jdbc.url}" />
    <property name="username" value="${jdbc.username}" />
    <property name="password" value="${jdbc.password}" />
    <property name="maxActive" value="10" />
    <property name="maxIdle" value="5" />
</bean>

<!-- 配置 SqlSessionFactory -->
<bean id="sqlSessionFactory"
class="org.mybatis.spring.SqlSessionFactoryBean">
    <!-- 数据库连接池 -->
    <property name="dataSource" ref="dataSource" />
    <!-- 加载 mybatis 的全局配置文件 -->
    <property name="configLocation"
value="classpath:mybatis/SqlMapConfig.xml" />
</bean>

<!-- 配置 Mapper 扫描 -->
<bean class="org.mybatis.spring.mapper.MapperScannerConfigurer">
    <!-- 配置 Mapper 扫描包 -->
    <property name="basePackage" value="cn.itcast.ssm.mapper" />
</bean>

</beans>
```

4.6.3. db.properties

配置数据库相关信息

```
jdbc.driver=com.mysql.jdbc.Driver
jdbc.url=jdbc:mysql://localhost:3306/springmvc?characterEncoding=utf-8
jdbc.username=root
jdbc.password=root
```



4.6.4. applicationContext-service.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:p="http://www.springframework.org/schema/p"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xmlns:tx="http://www.springframework.org/schema/tx"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context-4.0.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop-4.0.xsd
http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx-4.0.xsd
http://www.springframework.org/schema/util
http://www.springframework.org/schema/util/spring-util-4.0.xsd">

    <!-- 配置 service 扫描 -->
    <context:component-scan base-package="cn.itcast.ssm.service" />

</beans>
```

4.6.5. applicationContext-trans.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:p="http://www.springframework.org/schema/p"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xmlns:tx="http://www.springframework.org/schema/tx"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context-4.0.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop-4.0.xsd
http://www.springframework.org/schema/tx
http://www.springframework.org/schema/tx/spring-tx-4.0.xsd
http://www.springframework.org/schema/util
http://www.springframework.org/schema/util/spring-util-4.0.xsd">
```



```
<!-- 事务管理器 -->
<bean id="transactionManager"

    class="org.springframework.jdbc.datasource.DataSourceTransactionM
anager">
    <!-- 数据源 -->
    <property name="dataSource" ref="dataSource" />
</bean>

<!-- 通知 -->
<tx:advice id="txAdvice" transaction-manager="transactionManager">
    <tx:attributes>
        <!-- 传播行为 -->
        <tx:method name="save*" propagation="REQUIRED" />
        <tx:method name="insert*" propagation="REQUIRED" />
        <tx:method name="delete*" propagation="REQUIRED" />
        <tx:method name="update*" propagation="REQUIRED" />
        <tx:method name="find*" propagation="SUPPORTS"
read-only="true" />
        <tx:method name="get*" propagation="SUPPORTS"
read-only="true" />
        <tx:method name="query*" propagation="SUPPORTS"
read-only="true" />
    </tx:attributes>
</tx:advice>

<!-- 切面 -->
<aop:config>
    <aop:advisor advice-ref="txAdvice"
        pointcut="execution(* cn.itcast.ssm.service.*(..))" />
</aop:config>

</beans>
```

4.6.6. springmvc.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:p="http://www.springframework.org/schema/p"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:mvc="http://www.springframework.org/schema/mvc"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
```



```
http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
    http://www.springframework.org/schema/mvc
http://www.springframework.org/schema/mvc/spring-mvc-4.0.xsd
    http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context-4.0.xsd
">

<!-- 配置 controller 扫描包 -->
<context:component-scan base-package="cn.itcast.ssm.controller" />

<!-- 注解驱动 -->
<mvc:annotation-driven />

<!-- Example: prefix="/WEB-INF/jsp/", suffix=".jsp",
viewname="test" ->
    "/WEB-INF/jsp/test.jsp" -->
<!-- 配置视图解析器 -->
<bean
    class="org.springframework.web.servlet.view.InternalResourceViewR
esolver">
    <!-- 配置逻辑视图的前缀 -->
    <property name="prefix" value="/WEB-INF/jsp/" />
    <!-- 配置逻辑视图的后缀 -->
    <property name="suffix" value=".jsp" />
</bean>

</beans>
```

4.6.7. web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns="http://java.sun.com/xml/ns/javaee"
    xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
    id="WebApp_ID" version="2.5">
    <display-name>springmvc-web</display-name>
    <welcome-file-list>
        <welcome-file>index.html</welcome-file>
        <welcome-file>index.htm</welcome-file>
        <welcome-file>index.jsp</welcome-file>
        <welcome-file>default.html</welcome-file>
        <welcome-file>default.htm</welcome-file>
        <welcome-file>default.jsp</welcome-file>
```



```
</welcome-file-list>

<!-- 配置 spring -->
<context-param>
    <param-name>contextConfigLocation</param-name>

    <param-value>classpath:spring/applicationContext*.xml</param-value>
</context-param>

<!-- 使用监听器加载 Spring 配置文件 -->
<listener>

    <listener-class>org.springframework.web.context.ContextLoaderList
ener</listener-class>
</listener>

<!-- 配置 SpringMVC 的前端控制器 -->
<servlet>
    <servlet-name>springmvc-web</servlet-name>

    <servlet-class>org.springframework.web.servlet.DispatcherServlet<
/servlet-class>
    <init-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>classpath:spring/springmvc.xml</param-value>
    </init-param>
</servlet>

<servlet-mapping>
    <servlet-name>springmvc-web</servlet-name>
    <!-- 配置所有以 action 结尾的请求进入 SpringMVC -->
    <url-pattern>*.action</url-pattern>
</servlet-mapping>

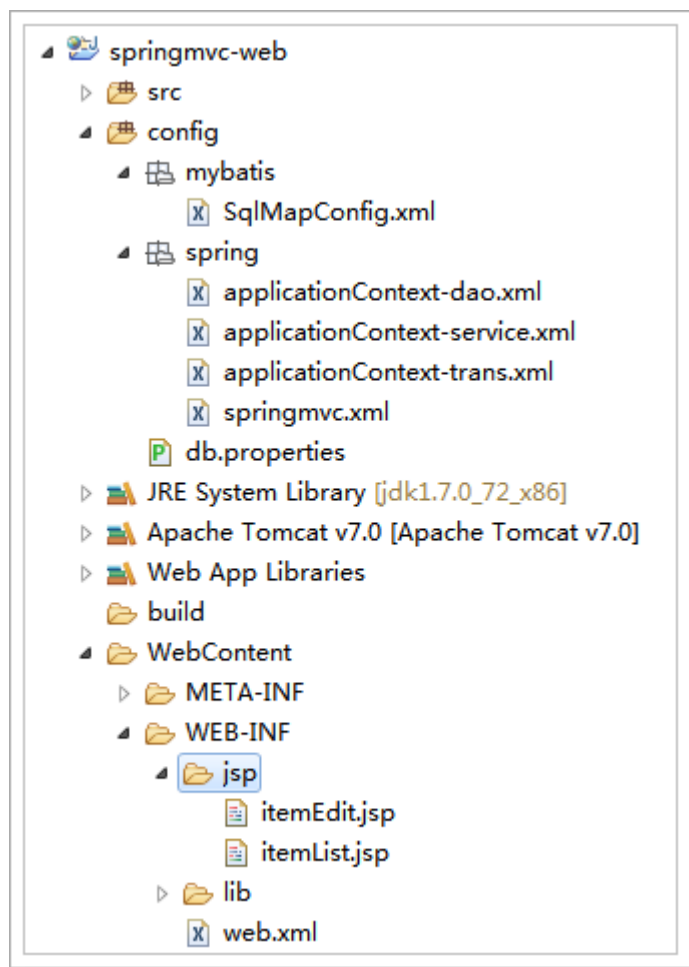
</web-app>
```

4.7. 加入 jsp 页面

复制课前资料的 itemList.jsp 和 itemEdit.jsp 到工程中

4.8. 效果

配置完效果如下图：



5. 实现商品列表显示

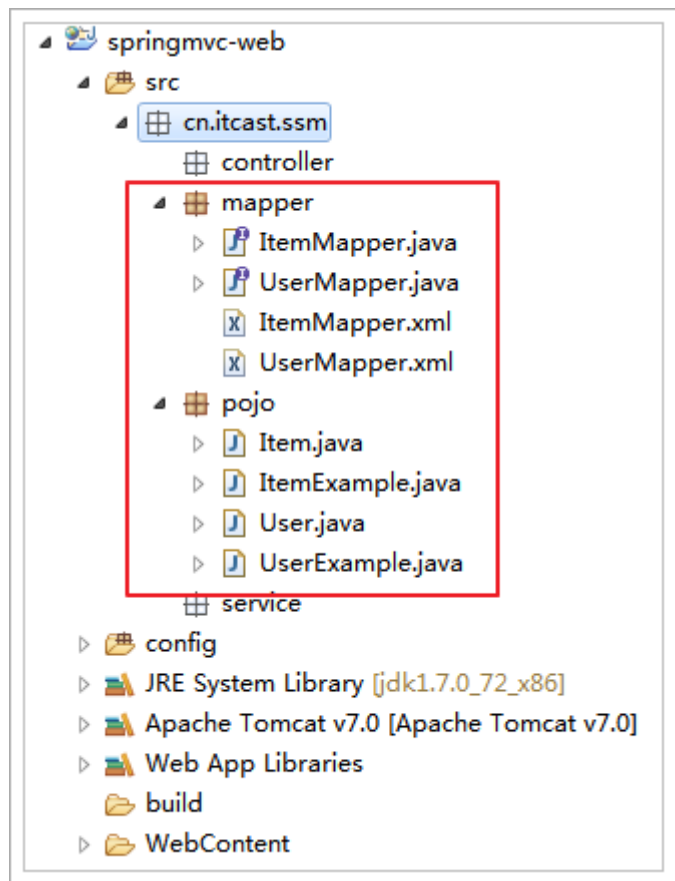
5.1. 需求

实现商品查询列表，从 mysql 数据库查询商品信息。

5.2. DAO 开发

使用逆向工程，生成代码

注意修改逆向工程的配置文件，参考 MyBatis 第二天
逆向工程生成代码如下图：



5.3. ItemService 接口

```
public interface ItemService {  
  
    /**  
     * 查询商品列表  
     *  
     * @return  
     */  
    List<Item> queryItemList();  
}
```



5.4. ItemServiceImpl 实现类

```
@Service
public class ItemServiceImpl implements ItemService {

    @Autowired
    private ItemMapper itemMapper;

    @Override
    public List<Item> queryItemList() {
        // 从数据库查询商品数据
        List<Item> list = this.itemMapper.selectByExample(null);

        return list;
    }
}
```

5.5. ItemController

```
@Controller
public class ItemController {

    @Autowired
    private ItemService itemService;

    /**
     * 显示商品列表
     *
     * @return
     */
    @RequestMapping("/itemList")
    public ModelAndView queryItemList() {
        // 获取商品数据
        List<Item> list = this.itemService.queryItemList();

        ModelAndView modelAndView = new ModelAndView();
        // 把商品数据放到模型中
        modelAndView.addObject("itemList", list);
        // 设置逻辑视图
        modelAndView.setViewName("itemList");

        return modelAndView;
    }
}
```



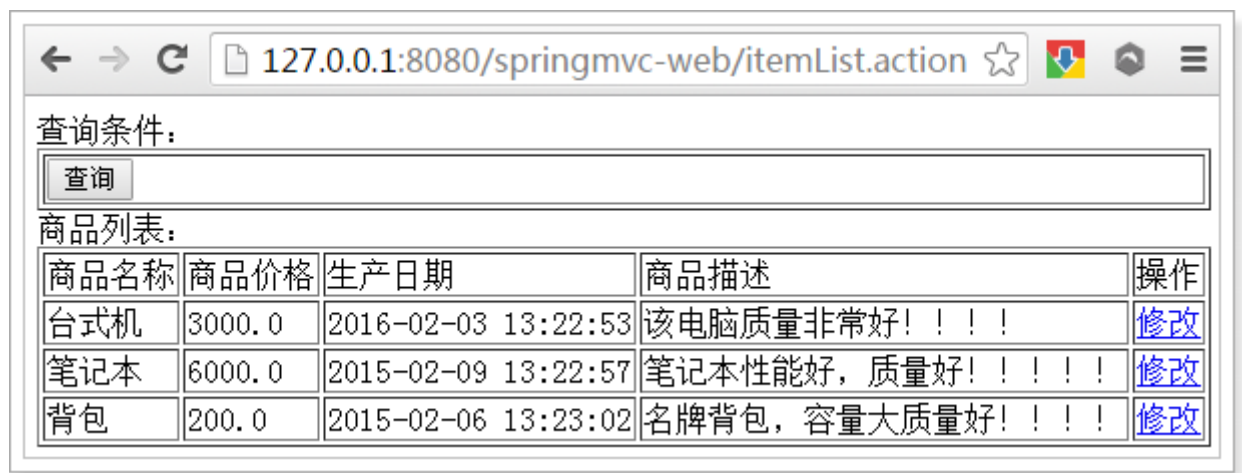
```
}  
  
}
```

5.6. 测试

访问 url:

http://127.0.0.1:8080/springmvc-web/itemList.action

效果如下图:



6. 参数绑定

6.1. 默认支持的参数类型

6.1.1. 需求

打开商品编辑页面，展示商品信息。

6.1.2. 需求分析

编辑商品信息，首先要显示商品详情

需要根据商品 id 查询商品信息，然后展示到页面。

请求的 url: /itemEdit.action

参数: id (商品 id)

响应结果: 商品编辑页面，展示商品详细信息。



6.1.3. ItemService 接口

编写 ItemService 接口如下图：



6.1.4. ItemServiceImpl 实现类

```
@Override
public Item queryItemById(int id) {
    Item item = this.itemMapper.selectByPrimaryKey(id);

    return item;
}
```

6.1.5. ItemController

页面点击修改按钮，发起请求

<http://127.0.0.1:8080/springmvc-web/itemEdit.action?id=1>

需要从请求的参数中把请求的 id 取出来。

Id 包含在 Request 对象中。可以从 Request 对象中取 id。

想获得 Request 对象只需要在 Controller 方法的形参中添加一个参数即可。
Springmvc 框架会自动把 Request 对象传递给方法。

代码实现

```
/**
 * 根据 id 查询商品
 *
 * @param request
 * @return
 */
```



```
@RequestMapping("/itemEdit")
public ModelAndView queryItemById(HttpServletRequest request) {
    // 从 request 中获取请求参数
    String strId = request.getParameter("id");
    Integer id = Integer.valueOf(strId);

    // 根据 id 查询商品数据
    Item item = this.itemService.queryItemById(id);

    // 把结果传递给页面
    ModelAndView modelAndView = new ModelAndView();
    // 把商品数据放在模型中
    modelAndView.addObject("item", item);
    // 设置逻辑视图
    modelAndView.setViewName("itemEdit");

    return modelAndView;
}
```

6.1.6. 默认支持的参数类型

处理器形参中添加如下类型的参数处理适配器会默认识别并进行赋值。

6.1.6.1. HttpServletRequest

通过 request 对象获取请求信息

6.1.6.2. HttpServletResponse

通过 response 处理响应信息

6.1.6.3. HttpSession

通过 session 对象得到 session 中存放的对象

6.1.7. Model/ModelMap

6.1.7.1. Model

除了 ModelAndView 以外，还可以使用 Model 来向页面传递数据，Model 是一个接口，在参数里直接声明 model 即可。

如果使用 Model 则可以不使用 ModelAndView 对象，Model 对象可以向页面传递数据，View 对象则可以使用 String 返回值替代。



不管是 Model 还是 ModelAndView, 其本质都是使用 Request 对象向 jsp 传递数据。
代码实现:

```
/**
 * 根据 id 查询商品,使用 Model
 *
 * @param request
 * @param model
 * @return
 */
@RequestMapping("/itemEdit")
public String queryItemById(HttpServletRequest request, Model model) {
    // 从 request 中获取请求参数
    String strId = request.getParameter("id");
    Integer id = Integer.valueOf(strId);

    // 根据 id 查询商品数据
    Item item = this.itemService.queryItemById(id);

    // 把结果传递给页面
    // ModelAndView modelAndView = new ModelAndView();
    // 把商品数据放在模型中
    // modelAndView.addObject("item", item);
    // 设置逻辑视图
    // modelAndView.setViewName("itemEdit");

    // 把商品数据放在模型中
    model.addAttribute("item", item);

    return "itemEdit";
}
```

6.1.7.2. ModelMap

ModelMap 是 Model 接口的实现类, 也可以通过 ModelMap 向页面传递数据

使用 Model 和 ModelMap 的效果一样, 如果直接使用 Model, springmvc 会实例化 ModelMap。

代码实现:

```
/**
 * 根据 id 查询商品,使用 ModelMap
 *
```



```
* @param request
* @param model
* @return
*/
@RequestMapping("/itemEdit")
public String queryItemById(HttpServletRequest request, ModelMap model)
{
    // 从 request 中获取请求参数
    String strId = request.getParameter("id");
    Integer id = Integer.valueOf(strId);

    // 根据 id 查询商品数据
    Item item = this.itemService.queryItemById(id);

    // 把结果传递给页面
    // ModelAndView modelAndView = new ModelAndView();
    // 把商品数据放在模型中
    // modelAndView.addObject("item", item);
    // 设置逻辑视图
    // modelAndView.setViewName("itemEdit");

    // 把商品数据放在模型中
    model.addAttribute("item", item);

    return "itemEdit";
}
```

6.2. 绑定简单类型

当请求的参数名称和处理器形参名称一致时会将请求参数与形参进行绑定。这样，从 Request 取参数的方法就可以进一步简化。

```
/**
 * 根据 id 查询商品,绑定简单数据类型
 *
 * @param id
 * @param model
 * @return
 */
@RequestMapping("/itemEdit")
public String queryItemById(int id, ModelMap model) {
    // 根据 id 查询商品数据
    Item item = this.itemService.queryItemById(id);
}
```



```
// 把商品数据放在模型中
model.addAttribute("item", item);

return "itemEdit";
}
```

6.2.1. 支持的数据类型

参数类型**推荐使用包装数据类型**，因为基础数据类型不可以为 null

整形：Integer、int

字符串：String

单精度：Float、float

双精度：Double、double

布尔型：Boolean、boolean

说明：对于布尔类型的参数，请求的参数值为 true 或 false。或者 1 或 0

请求 url:

http://localhost:8080/xxx.action?id=2&status=false

处理器方法:

```
public String editItem(Model model, Integer id, Boolean status)
```

6.2.2. @RequestParam

使用@RequestParam 常用于处理简单类型的绑定。

value: 参数名字，即入参的请求参数名字，如value= “itemId” 表示请求的参数区中的名字为itemId的参数的值将传入

required: 是否必须，默认是true，表示请求中一定要有相应的参数，否则将报错
TTP Status 400 - Required Integer parameter 'XXXX' is not present

defaultValue: 默认值，表示如果请求中没有同名参数时的默认值

定义如下:

```
@RequestMapping("/itemEdit")
public String queryItemById(@RequestParam(value = "itemId", required =
```



```
true, defaultValue = "1") Integer id,
    ModelMap modelMap) {
    // 根据 id 查询商品数据
    Item item = this.itemService.queryItemById(id);

    // 把商品数据放在模型中
    modelMap.addAttribute("item", item);

    return "itemEdit";
}
```

6.3. 绑定 pojo 类型

6.3.1. 需求

将页面修改后的商品信息保存到数据库中。

6.3.2. 需求分析

请求的 url: /updateItem.action

参数: 表单中的数据。

响应内容: 更新成功页面

6.3.3. 使用 pojo 接收表单数据

如果提交的参数很多, 或者提交的表单中的内容很多的时候, 可以使用简单类型接受数据, 也可以使用 pojo 接收数据。

要求: pojo 对象中的属性名和表单中 input 的 name 属性一致。

页面定义如下图:



Pojo(逆向工程生成)如下图:

```
public class Item {  
    private Integer id;  
  
    private String name;  
  
    private Float price;  
  
    private String detail;  
}
```

请求的参数名称和 pojo 的属性名称一致,会自动将请求参数赋值给 pojo 的属性。

6.3.4. ItemService 接口

ItemService 里编写接口方法

```
/**  
 * 根据 id 更新商品  
 *  
 * @param item  
 */  
void updateItemById(Item item);
```

6.3.5. ItemServiceImpl 实现类

ItemServiceImpl 里实现接口方法

使用 updateByPrimaryKeySelective(item) 方法, 忽略空参数

```
@Override  
public void updateItemById(Item item) {  
    this.itemMapper.updateByPrimaryKeySelective(item);  
}
```

6.3.6. ItemController

```
/**  
 * 更新商品,绑定 pojo 类型  
 *  
 * @param item  
 * @param model  
 * @return  
 */  
@RequestMapping("/updateItem")
```



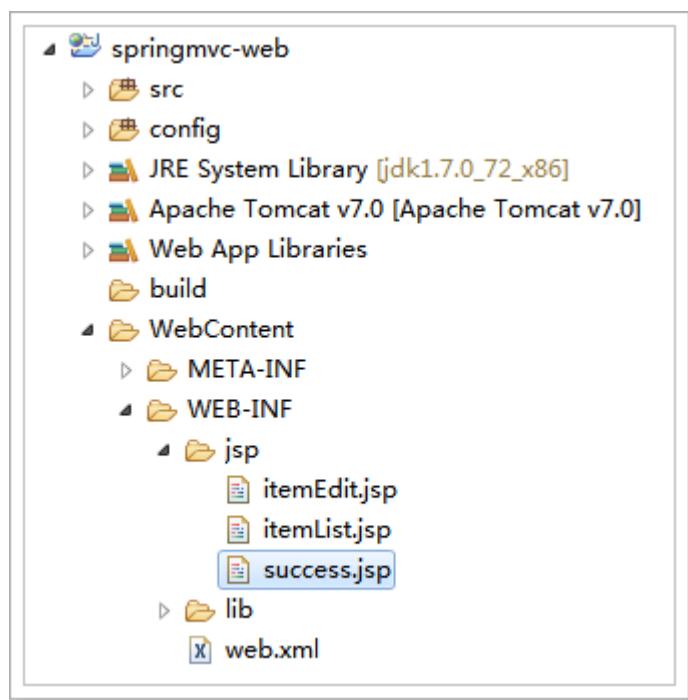
```
public String updateItem(Item item) {  
    // 调用服务更新商品  
    this.itemService.updateItemById(item);  
  
    // 返回逻辑视图  
    return "success";  
}
```

注意：

提交的表单中不要有日期类型的数据，否则会报 400 错误。如果想提交日期类型的数据需要用到后面的自定义参数绑定的内容。

6.3.7. 编写 success 页面

如下图创建 success.jsp 页面



页面代码：

```
<%@ page language="java" contentType="text/html; charset=UTF-8"  
    pageEncoding="UTF-8"%>  
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"  
    "http://www.w3.org/TR/html4/loose.dtd">  
<html>  
<head>  
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">  
<title>Insert title here</title>  
</head>
```



```
<body>

<h1>商品修改成功! </h1>

</body>
</html>
```

6.3.8. 解决 post 乱码问题

提交发现，保存成功，但是保存的是乱码

在 web.xml 中加入：

```
<!-- 解决 post 乱码问题 -->
<filter>
    <filter-name>encoding</filter-name>

    <filter-class>org.springframework.web.filter.CharacterEncodingFil
ter</filter-class>
    <!-- 设置编码参数是 UTF8 -->
    <init-param>
        <param-name>encoding</param-name>
        <param-value>UTF-8</param-value>
    </init-param>
</filter>
<filter-mapping>
    <filter-name>encoding</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>
```

以上可以解决 post 请求乱码问题。

对于 get 请求中文参数出现乱码解决方法有两个：

修改 tomcat 配置文件添加编码与工程编码一致，如下：

```
<Connector URIEncoding="utf-8" connectionTimeout="20000" port="8080"
protocol="HTTP/1.1" redirectPort="8443"/>
```

另外一种方法对参数进行重新编码：

```
String userName new
String(request.getParamter("userName").getBytes("ISO8859-1"),"utf-8")
```

ISO8859-1 是 tomcat 默认编码，需要将 tomcat 编码后的内容按 utf-8 编码



6.4. 绑定包装 pojo

6.4.1. 需求

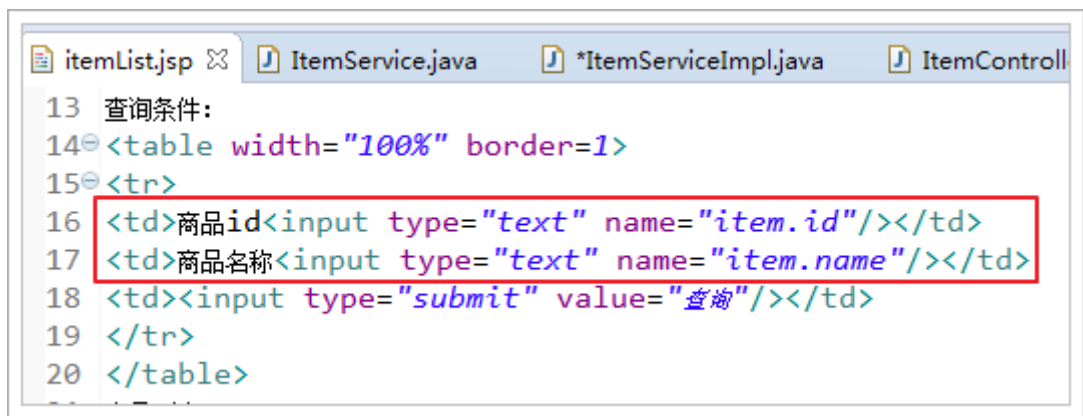
使用包装的 pojo 接收商品信息的查询条件。

6.4.2. 需求分析

包装对象定义如下：

```
public class QueryVo {  
    private Item item;  
    set/get...  
}
```

页面定义如下图：



6.4.3. 接收查询条件

```
// 绑定包装数据类型  
@RequestMapping("/queryItem")  
public String queryItem(QueryVo queryVo) {  
    System.out.println(queryVo.getItem().getId());  
    System.out.println(queryVo.getItem().getName());  
  
    return "success";  
}
```



6.5. 自定义参数绑定

6.5.1. 需求

在商品修改页面可以修改商品的生产日期，并且根据业务需求自定义日期格式。

6.5.2. 需求分析

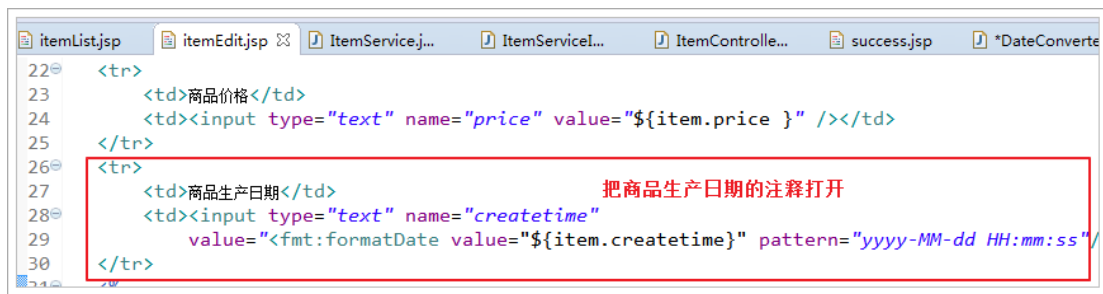
由于日期数据有很多种格式，springmvc 没办法把字符串转换成日期类型。所以需要自定义参数绑定。

前端控制器接收到请求后，找到注解形式的处理器适配器，对 RequestMapping 标记的方法进行适配，并对方法中的形参进行参数绑定。可以在 springmvc 处理器适配器上自定义转换器 Converter 进行参数绑定。

一般使用<mvc:annotation-driven/>注解驱动加载处理器适配器，可以在此标签上进行配置。

6.5.3. 修改 jsp 页面

如下图修改 itemEdit.jsp 页面，显示时间



6.5.4. 自定义 Converter

```
//Converter<S, T>
//S:source,需要转换的源的类型
//T:target,需要转换的目标类型
public class DateConverter implements Converter<String, Date> {

    @Override
    public Date convert(String source) {
        try {
```



```
// 把字符串转换为日期类型
SimpleDateFormat simpleDateFormat = new
SimpleDateFormat("yyy-MM-dd HH:mm:ss");
Date date = simpleDateFormat.parse(source);

return date;
} catch (ParseException e) {
    // TODO Auto-generated catch block
    e.printStackTrace();
}
// 如果转换异常则返回空
return null;
}
}
```

6.5.5. 配置 Converter

我们同时可以配置多个的转换器。

类似下图的 usb 设备，可以接入多个 usb 设备



```
<!-- 配置注解驱动 -->
<!-- 如果配置此标签,可以不用配置... -->
<mvc:annotation-driven conversion-service="conversionService" />

<!-- 转换器配置 -->
<bean id="conversionService"
class="org.springframework.format.support.FormattingConversion
ServiceFactoryBean">
    <property name="converters">
        <set>
```



```
        <bean  
class="cn.itcast.springmvc.converter.DateConverter" />  
        </set>  
    </property>  
</bean>
```

6.5.6. 配置方式 2（了解）

```
<!-- 注解适配器 -->  
<bean  
class="org.springframework.web.servlet.mvc.method.annotation.R  
equestMappingHandlerAdapter">  
    <property name="webBindingInitializer"  
ref="customBinder"></property>  
</bean>  
  
<!-- 自定义webBinder -->  
<bean id="customBinder"  
class="org.springframework.web.bind.support.ConfigurableWebBin  
dingInitializer">  
    <property name="conversionService" ref="conversionService" />  
</bean>  
  
<!-- 转换器配置 -->  
<bean id="conversionService"  
class="org.springframework.format.support.FormattingConversion  
ServiceFactoryBean">  
    <property name="converters">  
        <set>  
            <bean  
class="cn.itcast.springmvc.convert.DateConverter" />  
        </set>  
    </property>  
</bean>
```

注意：此方法需要独立配置处理器映射器、适配器，
不再使用<mvc:annotation-driven/>



7. springmvc 与 struts2 不同

- 1、springmvc 的入口是一个 servlet 即前端控制器，而 struts2 入口是一个 filter 过滤器。
- 2、springmvc 是基于方法开发(一个 url 对应一个方法)，请求参数传递到方法的形参，可以设计为单例或多例(建议单例)，struts2 是基于类开发，传递参数是通过类的属性，只能设计为多例。
- 3、Struts 采用值栈存储请求和响应的数据，通过 OGNL 存取数据，springmvc 通过参数解析器是将 request 请求内容解析，并给方法形参赋值，将数据和视图封装成 ModelAndView 对象，最后又将 ModelAndView 中的模型数据通过 request 域传输到页面。Jsp 视图解析器默认使用 jstl。